# Sanskrit as a Stable "Eigen"-Language Well-Suited for Storage and Transmission

Aman Chawla

April 14, 2023

**Abstract**

In this paper, the authors examine the possibility that Sanskrit is an eigen-language of the transformation which takes in sentences in a given language, and generates other sentences by permuting the components of the input sentence. Under this setting, if we are given one more, or can construct one more, eigen-language of this transformation, then we should be able to generate an entire eigenspace of languages like Sanskrit. Further, given the central role of eigen-analysis in linear algebra and the theorems related to it, it is likely that this paper suggests the way for a deep mathematical study of Sanskrit and related language groups.

## 1 Introduction

Old Indo-Aryan has been preserved for many centuries in India. It is thought that the key text in the language, the Vedas, were preserved orally over many generations by oral transmission within families. This leads us to several key questions about Sanskrit:

- Is there something inherent in the language that protects it from transmission errors?
- Were its speakers aware of its special character?
- Did they use its special character in the design of its texts?

While we do not approach an answer to all of these questions, we take some initial steps in this paper. Specifically, we are able to suggest a view of Sanskrit as 'stable' in some specific mathematical sense. If developed further, this hypothesis could have ramifications for the study of Old Indo-Aryan, and also boost the claims that Sanskrit might be useful in modern computing.

This paper is structured as follows. In Section 2 we present the mathematical framework for our analysis of Sanskrit. In Section 3 we apply this framework to Sanskrit. In Section 4 we conclude.

# 2 Mathematical Framework

## 2.1 Review of Permutation Matrices

A permutation matrix is a square matrix that represents a permutation of the rows or columns of an identity matrix. In other words, a permutation matrix is a matrix that results from swapping the rows or columns of an identity matrix. An identity matrix is a square matrix where all the diagonal elements are equal to 1 and all the off-diagonal elements are equal to 0. A permutation matrix has the same properties as an identity matrix, but with its rows or columns permuted.

The rows or columns of a permutation matrix represent the new order of the rows or columns of the original matrix. For example, if we have a 3x3 permutation matrix P, and we multiply it by a 3x3 matrix A, the result is a new matrix B whose rows are the rows of A reordered according to the permutation matrix P.

Permutation matrices are used in linear algebra, particularly in matrix multiplication, matrix inversion, and matrix factorization. They also have applications in combinatorics, graph theory, and cryptography.

**Example 1.** A Permutation Matrix P

Suppose we have a 3x3 matrix

$$A = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] \tag{1}$$

and we have a 3x3 permutation matrix P that swaps the first and third rows of the identity matrix:

$$P = [[0, 0, 1], [0, 1, 0], [1, 0, 0]] \tag{2}$$

.

To apply the permutation matrix to matrix A, we simply multiply A by P to get B = PA. The resulting matrix B will have its rows permuted according to the permutation matrix P. In this case, we get

$$B = [[7, 8, 9], [4, 5, 6], [1, 2, 3]] \tag{3}$$

.

We can see that the first and third rows of A have been swapped in B, as specified by the permutation matrix P. Note that the columns of A have not been affected, since P is a permutation of the rows of the identity matrix.

Permutation matrices can also be used to permute the columns of a matrix. To do this, we simply multiply the matrix by a permutation matrix on the right, instead of on the left.

## 2.2 Sentences as Vectors

Suppose we take a vector, and for its entries, use the words in a sentence S that is given. To do this, we first need to tokenize the sentence, that is, split it into individual words.

For example, suppose the sentence S is "The quick brown fox jumps over the lazy dog". To represent this sentence as a vector, we can first tokenize it as follows:

["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]

This representation of a sentence as a vector can be used in various natural language processing tasks, such as document classification, information retrieval, and text summarization.

## 2.3 Sentence-Vectors as Inputs to Permutation Matrices

After tokenization, instead of counting frequencies, we can use the tokenized vector as such as our 'input' vector into a 'permutation' matrix. We are essentially permuting the order of the words in the sentence. A permutation matrix is a square matrix that represents a permutation of a set of elements, in this case the words in the sentence.

To apply a permutation matrix to the tokenized vector, we can treat the vector as a row vector and multiply it on the left by the permutation matrix. This will result in a new vector whose entries are the words of the sentence arranged in a different order.

For example, suppose the tokenized vector for the sentence S is:

["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]

and suppose we have a permutation matrix P that corresponds to the permutation (2 4 6 1 3 5 8 7 9), which swaps the second word with the fourth, the sixth word with the first, and so on.

To apply the permutation matrix to the tokenized vector, we can treat the vector as a row vector and multiply it on the left by the permutation matrix P:

$$[[\text{"}The\text{"},\text{"}quick\text{"},\text{"}brown\text{"},\text{"}fox\text{"},\text{"}jumps\text{"},\text{"}over\text{"},\text{"}the\text{"},\text{"}lazy\text{"},\text{"}dog\text{"}]]P = \tag{4}$$

$$[[\text{"}quick\text{"},\text{"}fox\text{"},\text{"}over\text{"},\text{"}The\text{"},\text{"}brown\text{"},\text{"}jumps\text{"},\text{"}lazy\text{"},\text{"}the\text{"},\text{"}dog\text{"}]] \tag{5}$$

The resulting vector has the words of the sentence rearranged according to the permutation specified by the permutation matrix.

Note that the permutation matrix must be a square matrix with the same number of rows and columns as the length of the tokenized vector. Additionally, to ensure that the permutation matrix represents a valid permutation, each row and each column of the matrix must have exactly one entry that is equal to 1, and all other entries must be 0.

## 2.4 Visual Representation of Vectors

If we take the tokenized vector, [["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]], and draw a visual graphic representing the image suggested by the vector, it could take many forms depending on how you choose to represent the vector.

That is, we could represent the sentence by a painting based on the visual image it suggests. So, for [["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]], we would paint a brown colored fox-like shape over a grey colored (say) dog-like shape resting on a surface.

This idea of painting a brown colored fox-like shape over a grey colored dog-like shape resting on a surface is one way to visually represent the sentence, and it is open to interpretation based on the artist's creativity and imagination. This approach to representing language through visual art is often used in various forms of literature, such as poetry and visual storytelling.

In order to create such a painting, we would need to first decide on the specific style, color palette, and composition that we want to use to convey the visual image of the sentence. We could use various artistic techniques such as painting, drawing, or mixed media to create the image, and we may also want to experiment with different textures, shapes, and perspectives to add depth and interest to the painting.

Ultimately, the final painting would be a unique representation of the sentence that captures the artist's personal interpretation and style, and it would be open to interpretation and discussion by the viewer.

## 2.5 Standardization

However, herein we ask if this process could be 'standardized' so that, for example, a particular sentence always produces the same painting. It would require defining a set of rules and parameters for creating the painting that are consistent and reproducible.

One approach to standardizing this process could be to create a specific set of visual elements, such as colors, shapes, and textures, that are assigned to each word in the sentence. For example, "fox" could always be represented by a red-orange color, a triangular shape, and a furry texture, while "dog" could always be represented by a gray-brown color, a round shape, and a smooth texture.

Using these predefined visual elements, we could create a standardized set of rules for arranging and combining the elements to create a painting that represents the sentence. For example, we could always place the "fox" element above the "dog" element, or we could use a specific background color or texture to represent the environment in which the animals are situated.

However, even with a standardized set of rules and visual elements, there would still be room for interpretation and creativity in how the elements are arranged and combined to create the painting. Different artists may choose to emphasize certain elements or use different techniques to convey the visual

image of the sentence, resulting in variations in the final painting.

### 2.6 Advantage of Standardization

The benefit of standardization is that two different sentences will produce two different paintings, always. Standardization can help ensure that each sentence produces a unique and distinguishable painting, which can be useful for applications such as visualizing text data or creating a visual representation of language.

By defining a standardized set of visual elements and rules for combining them, we can create a consistent and reproducible method for translating sentences into paintings. This can also help facilitate comparison and analysis of different sentences based on their visual representation.

However, it's important to note that standardization doesn't necessarily mean that the paintings will be identical or lack artistic expression. Even within a standardized framework, there is still room for creativity and variation in how the visual elements are arranged and combined to create the final painting. Each artist may interpret and represent the sentence in their own unique way, resulting in subtle differences between paintings.

**Example 2.** Painting With P

So, next, we will permute [["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]] as in Equation (4). Ideally, we should have two different paintings, one for the input sentence and one for the output sentence. Permuting the tokenized vector and using the resulting permutation matrix as the input for creating a painting can produce a different painting than the original sentence. This is because the permutation matrix changes the order of the tokens in the vector, which can alter the visual image that the sentence suggests.

For another example, if we apply a permutation matrix to the original tokenized vector [["The", "quick", "brown", "fox", "jumps", "over", "the", "lazy", "dog"]], we might get a new vector such as [["jumps", "brown", "quick", "dog", "over", "fox", "lazy", "The", "the"]]. This new vector has a different order of tokens than the original vector, which can suggest a different visual image and thus result in a different painting.

Creating a painting for both the original sentence and the permuted sentence can help us visualize and compare the differences between them.

## 3 The Case of Sanskrit

### 3.1 Introduction

In most languages, if we permute a sentence, it conveys a different meaning (take the English example we have just discussed). But is it the same in Sanskrit? In Sanskrit, as in any language, permuting the order of words in a sentence can change the meaning or emphasis of the sentence. However, the degree to which the meaning changes may vary depending on the specific sentence and context.

Sanskrit is a highly inflected language, which means that word order is relatively flexible compared to languages like English. In Sanskrit, the grammatical relationships between words are often indicated by case endings and other inflections, rather than by their position in the sentence. This means that changing the order of words may not always result in a significant change in meaning, as long as the grammatical relationships between the words remain intact.

In general, the degree to which meaning changes when permuting a sentence will depend on the specific sentence, context, and the language in question.

**Example 3.** Rama and Ravana

This example is about how we would write Rama killed Ravana, in Sanskrit. In Sanskrit, "Rama killed Ravana" can be written as (rāmaḥ rāvaṇaṃ hatvā). Here, (rāmaḥ) means "Rama," (rāvaṇaṃ) means "Ravana," and (hatvā) means "killed." The word order in Sanskrit is relatively flexible, so this sentence could also be written as (rāmaḥ hatvā rāvaṇaṃ) without significantly changing the meaning.

Consequently, if we passed rāmaḥ rāvaṇaṃ hatvā through our permutation 'engine' P, the painting produced would be the same (within minor artistic changes) as if we didn't pass it through the engine. The resulting tokenized vector would be the same as the original sentence, and the painting produced by the engine would be the same as the painting produced for the original sentence (again, within minor artistic changes).

This is because in this particular sentence, the order of the words does not significantly affect the meaning, and the grammatical relationships between the words are clear even if the word order is changed.

## 3.2 Sanskrit-Eigenvectors

In the context of our permutation engine, the tokenized vector representation of a Sanskrit sentence can be considered as an input vector, and the resulting painting can be considered as the output vector. In this sense, each sentence can be seen as mapping to a unique output vector (or painting), just like each eigenvector maps to a scaled version of itself under a linear transformation.

It's worth noting, though, that the relationship between the input and output vectors for a given sentence is not necessarily linear, as the permutation matrix is not a linear transformation. Additionally, the output paintings may not be exact replicas of each other, even for the same sentence, since there may be some variability introduced by the artistic interpretation of the output.

## 3.3 Non-Linearity of Permutation Matrices

One way to see that the permutation matrix is not a linear transformation is to consider its effect on two input vectors, say $u$ and $v$, and compare it to the effect of a linear transformation $T$ on the same two vectors.

Let $P$ be the permutation matrix, and let $u$ and $v$ be two vectors. Then, we have:

$$P(u + v) = P(u) + P(v) \tag{6}$$

However, this property does not hold for all matrices $P$ since addition of two tokenized sentences is not well-defined so far. In general, for a linear transformation $T$, we have:

$$T(u + v) = T(u) + T(v) \tag{7}$$

So, the permutation matrix does not satisfy the additivity property that is required for a matrix to be a linear transformation. Instead, it satisfies the properties of a permutation, which is a bijective mapping that preserves the cardinality of sets.

Furthermore, the permutation matrix is not scalar-multiplicative, which is another property that is required for a matrix to be a linear transformation. That is, for any scalar c and vector u, we have:

$$P(cu) = cP(u) \tag{8}$$

However, this property does not hold for the permutation matrix, as multiplying a vector by a scalar does not affect the relative positions of its entries. Therefore, we can conclude that the permutation matrix is not a linear transformation.

### 3.4 Approximate Linear Transformation

Furthermore, it is not possible to approximate the permutation matrix by a linear transformation in the sense that a linear transformation cannot replicate the unique behavior of a permutation matrix.

A permutation matrix is a matrix that permutes the rows or columns of another matrix. It is a very specific type of matrix that is defined by the permutation it represents. On the other hand, a linear transformation is a more general concept that can be represented by any matrix.

Because a permutation matrix is a very specific type of matrix, it has unique properties that cannot be replicated by a linear transformation. For example, a permutation matrix is always invertible, and its inverse is simply its transpose. This property does not hold for all matrices, and it is not a property of linear transformations in general.

In general, it is not possible to approximate the unique behavior of a permutation matrix by a linear transformation, since a linear transformation cannot replicate its specific properties. However, it is possible to use linear algebra techniques to analyze the behavior of a permutation matrix, such as finding its eigenvalues and eigenvectors, or computing its determinant and rank.

### 3.5 Linear Algebra on Permutation Matrices

Finding the eigenvalues and eigenvectors of a permutation matrix can be done using several linear algebra methods. A common method would be matrix

decomposition wherein a permutation matrix can be viewed as a special case of a matrix with only 0s and 1s. Therefore, one can use matrix decomposition techniques, such as LU decomposition, to find the eigenvalues and eigenvectors of a permutation matrix. In this case, the eigenvalues can be found as the roots of the characteristic polynomial, and the eigenvectors can be found by solving a system of linear equations.

### 3.6 Higher-Dimensional Spaces

If we represent tokenized Sanskrit sentences as standardized visual paintings, then we can treat them as vectors in a high-dimensional space. In this context, we can view the permutation engine as a linear transformation that takes one painting (or vector) and maps it to another. However, since the permutation matrix is not a linear transformation, the concept of eigenvectors and eigenvalues does not directly apply to it.

One alternate way to potentially relate the concept of eigenvectors to the permutation engine is to consider the idea of stability under the permutation transformation. Specifically, if a painting is stable under the permutation transformation, meaning that the output of the transformation is the same as the input, then we can view it as an "eigenvector" of sorts. That is, the painting is unchanged by the permutation transformation, much like an eigenvector is unchanged by a linear transformation.

This highlights the idea that certain paintings may be more stable under the permutation transformation than others, and this stability can be seen as a desirable property that is analogous to the concept of eigenvectors in linear algebra.

## 4 Concluding Discussion: Sanskrit- Versus French-Based Paintings

Given the suggested stability analysis of Subsection 3.6, Sanskrit based paintings would be more stable, than say French based paintings. However, it's difficult to make a definitive statement about the stability of paintings based on the language of the original sentence, as it would depend on the specific permutation engine being used and how it maps inputs to outputs. Nevertheless, in general, the more structure and regularity there is in the input painting, the more likely it is to be stable under the permutation transformation.

In the case of Sanskrit, which has a highly structured grammar and syntax, the paintings based on its sentences may have more regularity and structure than those based on a language with a less structured grammar. This could make them more stable under the permutation transformation and potentially more akin to "eigenvectors" in the sense described earlier. However, this is only a hypothesis, and it would depend on the specific details of the permutation engine and the types of transformations being applied.

Once we have shown Sanskrit as an eigenlanguage in this sense, it can also be considered to be more robust to certain types of transmission errors when sent over communication channels. In fact, this may be a contributing factor in the preservation of the Vedas as the oldest living scripture by human-to-human oral transmission over many generations. This conclusion has implications for our understanding of Old Indo-Aryan and its speakers.

## Acknowledgments

The author thanks OpenAI for access to the language model ChatGPT.